Data Structure Design - Interview Cheatsheet

Custom Stack | LRU Cache | Min Stack | Circular Queue | Queue/Stack Conversions

1 Design Stack with Increment Operation (LC 1381)

Problem: Design a stack with:

- push(x): Add element (if not full)
- pop(): Remove and return top element
- increment(k, val): Add val to bottom k elements
- All operations in O(1) except increment

Solution: Array-Based Stack

```java
class CustomStack {
    private int[] arr;          // Space: O(maxSize)
    private int top;            // Current top index
    private int maxSize;        // Capacity

    public CustomStack(int maxSize) {  // Time: O(maxSize), Space: O(maxSize)
        this.arr = new int[maxSize];
        this.top = -1;  // Empty stack
        this.maxSize = maxSize;
    }

    public void push(int x) {  // Time: O(1), Space: O(1)
        if (top < maxSize - 1) {  // Check if not full
            top++;
            arr[top] = x;
        }
    }

    public int pop() {  // Time: O(1), Space: O(1)
        if (top == -1) return -1;  // Check if empty
        return arr[top--];  // Return and decrement
    }

    public void increment(int k, int val) {  // Time: O(min(k,n)), Space: O(1)
        int limit = Math.min(k, top + 1);
        for (int i = 0; i < limit; i++) {
            arr[i] += val;
        }
    }
}
```

Tips & Tricks

Key Insights:

- **top = -1**: Elegant empty state, first push becomes index 0
- **Boundary checks**: Always check full/empty before operations
- **Array indexing**: arr[top] is current top, arr[0..k-1] are bottom k
- **Why not ArrayList?** Fixed size requirement + O(1) access needed

Common Mistakes

- if (top < maxSize) → Use top < maxSize - 1 (array is 0-indexed)
- Forgetting this. in constructor → Variables become local
- increment(k, val) without Math.min → ArrayIndexOutOfBounds
- Not checking empty in pop() → Returns garbage value

Time & Space Complexity

Operation	Time	Space
Constructor	O(maxSize)	O(maxSize)
push(x)	O(1)	O(1)
pop()	O(1)	O(1)
increment(k, val)	O(min(k, n))	O(1)

Dry Run: CustomStack(3), push(1), push(2), pop(), push(2), push(3), push(4), increment(5,100), increment(2,100), pop()

Constructor: arr=[_,_,_], top=-1
push(1): arr=[1,_,_], top=0
push(2): arr=[1,2,_], top=1
pop(): return 2, top=0
push(2): arr=[1,2,_], top=1
push(3): arr=[1,2,3], top=2
push(4): FULL! No change
increment(5,100): arr=[101,102,103], top=2

increment(2,100): arr=[201,202,103], top=2
pop(): return 103, top=1

2 LRU Cache (LC 146) - Amazon Favorite

Problem: Design a cache with fixed capacity:

- get(key): Return value (mark as recently used)
- put(key, value): Insert/update (evict LRU if full)
- Both operations must be O(1)!

Solution: HashMap + Doubly Linked List

```java
class Node {
    int key, val;
    Node prev, next;
    public Node(int key, int val) {
        this.key = key; this.val = val;
    }
}

class LRUCache {
    private int cap;  // Space: O(1)
    private HashMap<Integer, Node> cache;  // Space: O(capacity)
    private Node left, right;  // Dummy nodes, Space: O(1)

    public LRUCache(int capacity) {  // Time: O(1), Space: O(capacity)
        this.cap = capacity;
        this.cache = new HashMap<>();
        this.left = new Node(0, 0);    // LRU side (dummy)
        this.right = new Node(0, 0);   // MRU side (dummy)
        this.left.next = this.right;
        this.right.prev = this.left;
    }

    private void remove(Node node) {  // Time: O(1), Space: O(1)
        Node prev = node.prev, nxt = node.next;
        prev.next = nxt;  // Skip node
        nxt.prev = prev;
    }

    private void insert(Node node) {  // Time: O(1), Space: O(1)
        Node prev = this.right.prev;  // Insert before right dummy
        prev.next = node;
        node.prev = prev;
        node.next = this.right;
        this.right.prev = node;
    }

    public int get(int key) {  // Time: O(1), Space: O(1)
        if (cache.containsKey(key)) {
            Node node = cache.get(key);
            remove(node);    // Remove from current position
            insert(node);    // Insert at MRU position
            return node.val;
        }
        return -1;
    }

```

```
    public void put(int key, int value) {  // Time: O(1), Space: O(1)
        if (cache.containsKey(key)) {
            remove(cache.get(key));  // Remove old version
        }
        Node newNode = new Node(key, value);
        cache.put(key, newNode);
        insert(newNode);  // Insert at MRU

        if (cache.size() > cap) {  // Evict LRU
            Node lru = this.left.next;
            remove(lru);
            cache.remove(lru.key);
        }
    }
}
```

Tips & Tricks

Key Insights:

- **Dummy nodes**: Eliminate null checks, always have head/tail
- **Why HashMap + List?** HashMap=O(1) lookup, List=O(1) insert/delete
- **Right = MRU, Left = LRU**: Recently used moves to right
- **No actual deletion**: Just move pointers, let GC handle memory

Common Mistakes

- Missing node.next = this.right in insert() → NullPointerException
- Using PriorityQueue → remove() is O(n), fails requirement
- Forgetting cache.remove(lru.key) → Memory leak
- Not removing before inserting in put() → Duplicates in list

Time & Space Complexity

Operation	Time	Space
Constructor	O(1)	O(capacity)
get(key)	O(1)	O(1)
put(key, value)	O(1)	O(1)
remove(node)	O(1)	O(1)
insert(node)	O(1)	O(1)

Dry Run: LRUCache(2), put(1,1), put(2,2), get(1), put(3,3)

Constructor: [left] <-> [right], cache={}
put(1,1): [left] <-> [1:1] <-> [right]
put(2,2): [left] <-> [1:1] <-> [2:2] <-> [right]
get(1): [left] <-> [2:2] <-> [1:1] <-> [right] (1 moved)
put(3,3): [left] <-> [1:1] <-> [3:3] <-> [right] (2 evicted)
cache = {1: Node@A, 3: Node@C}

Visualization:

HashMap Doubly Linked List
{1: Node@A, 3: Node@C} [L] <-> [1:1] <-> [3:3] <-> [R]
| | LRU MRU
+-----+-----+
|
Same Node objects (shared references)

3 Min Stack - Approach 1: Pairs (LC 155)

Problem: Design a stack supporting:

- push(x), pop(), top(): Standard operations
- getMin(): Return minimum element in O(1)
- All operations must be O(1)!

Solution: Store [value

```java
class MinStack {
    private Stack<int[]> stack;  // Each element: [value, currentMin]
                                 // Space: O(n) where n = # of elements

    public MinStack() {  // Time: O(1), Space: O(1)
        this.stack = new Stack<>();
    }

    public void push(int val) {  // Time: O(1), Space: O(1)
        if (stack.isEmpty()) {
            stack.push(new int[]{val, val});  // First element
        } else {
            int currentMin = Math.min(val, stack.peek()[1]);
            stack.push(new int[]{val, currentMin});
        }
    }

    public void pop() {  // Time: O(1), Space: O(1)
        stack.pop();
    }

    public int top() {  // Time: O(1), Space: O(1)
        return stack.peek()[0];  // Access value part
    }

    public int getMin() {  // Time: O(1), Space: O(1)
        return stack.peek()[1];  // Access min part
    }
}
```

Tips & Tricks

Key Insights:

- **Store history**: Each element "remembers" what min was when pushed
- **No separate structure**: Min tracking built into each element
- **Pop is trivial**: Just remove top, previous min is in new top
- **Space trade-off**: 2 integers per element vs 1

Common Mistakes

- Using PriorityQueue for tracking min → pop() breaks heap structure
- stack.peek()[2] → ArrayIndexOutOfBounds (only 0 and 1)
- Forgetting isEmpty() check → NullPointerException
- Not using Math.min → Wrong min tracking

Time & Space Complexity

Operation	Time	Space
Constructor	O(1)	O(1)
push(val)	O(1)	O(1) per element
pop()	O(1)	O(1)
top()	O(1)	O(1)
getMin()	O(1)	O(1)
Total Space	-	**O(n)**

Dry Run: push(-2), push(0), push(-3), getMin(), pop(), top(), getMin()

push(-2): [[-2, -2]] min = -2
push(0): [[-2, -2], [0, -2]] min = -2
push(-3): [[-2, -2], [0, -2], [-3, -3]] min = -3
getMin(): return -3
pop(): [[-2, -2], [0, -2]] min = -2 (auto-restored!)
top(): return 0
getMin(): return -2

Visualization:

Stack: [[value, min], [value, min], ...]

After push(-2), push(0), push(-3):

```
+--------+
| -3, -3 | <- top, min = -3
+--------+
| 0,  -2 |
+--------+
| -2, -2 |
+--------+
```

After pop():

```
+--------+
```

```
| 0,  -2 | <- top, min = -2 (restored!)
+--------+
| -2, -2 |
+--------+
```

3 Min Stack - Approach 2: Two Stacks (LC 155)

Same problem, different approach using separate min tracking.

Solution: Main Stack + Min Stack

```java
class MinStack {
    private Stack<Integer> stack;      // Main stack, Space: O(n)
    private Stack<Integer> minStack;   // Tracks minimums, Space: O(n) worst

    public MinStack() {  // Time: O(1), Space: O(1)
        this.stack = new Stack<>();
        this.minStack = new Stack<>();
    }

    public void push(int val) {  // Time: O(1), Space: O(1)
        stack.push(val);
        // Push to minStack if new minimum or equal (handle duplicates)
        if (minStack.isEmpty() || val <= minStack.peek()) {
            minStack.push(val);
        }
    }

    public void pop() {  // Time: O(1), Space: O(1)
        int popped = stack.pop();
        // If popping current min, remove from minStack too
        if (popped == minStack.peek()) {
            minStack.pop();
        }
    }

    public int top() {  // Time: O(1), Space: O(1)
        return stack.peek();
    }

    public int getMin() {  // Time: O(1), Space: O(1)
        return minStack.peek();  // Top of minStack is current min
    }
}
```

Tips & Tricks

Key Insights:

- **Parallel stacks**: minStack mirrors main stack but only for minimums
- **val <= minStack.peek()**: Use <= not < (handles duplicates!)
- **Space efficient**: minStack often much smaller than main stack
- **Sync on pop**: Remove from minStack only if popping the current min

Common Mistakes

- Using val < minStack.peek() → Fails with duplicate minimums
- Always popping from minStack → Breaks min tracking
- if (popped == minStack.peek()) without null check → NPE
- Comparing with .equals() for primitives → Use ==

Time & Space Complexity

Operation	Time	Space
Constructor	O(1)	O(1)
push(val)	O(1)	O(1) per element
pop()	O(1)	O(1)
top()	O(1)	O(1)
getMin()	O(1)	O(1)
Total Space	-	**O(n)**

Note: minStack size varies: O(1) best case (increasing values), O(n) worst case (decreasing values)
Dry Run: push(-2), push(0), push(-3), getMin(), pop(), getMin()

push(-2):
  stack: [-2] minStack: [-2]
push(0):
  stack: [-2, 0] minStack: [-2] (0 > -2, don't push)
push(-3):
  stack: [-2, 0, -3] minStack: [-2, -3] (-3 <= -2, push)
getMin():
  return -3 (top of minStack)
pop():
  popped = -3, minStack.peek() = -3, pop from both
  stack: [-2, 0] minStack: [-2]
getMin():
  return -2

Visualization:

After push(-2), push(0), push(-3):

Main Stack: Min Stack:
+----+ +----+
| -3 | <- top | -3 | <- current min
+----+ +----+
| 0 | | -2 |
+----+ +----+
| -2 | +----+
+----+

After pop():
+----+ +----+
| 0 | <- top | -2 | <- current min
+----+ +----+
| -2 |
+----+

Space Analysis:

Best case (increasing): [1,2,3,4,5]
  stack: [1,2,3,4,5] (5 elements)
  minStack: [1] (1 element) -> O(1)

Worst case (decreasing): [5,4,3,2,1]
  stack: [5,4,3,2,1] (5 elements)
  minStack: [5,4,3,2,1] (5 elements) -> O(n)

Average: O(log n) for random values

4 Design Circular Queue (LC 622)

Problem: Design fixed-size queue with:

- enQueue(value): Add to rear (return false if full)
- deQueue(): Remove from front
- Front(), Rear(): Get front/rear value
- All operations must be O(1)!

Solution: Array + Modulo Arithmetic

```java
class MyCircularQueue {
    private int[] queue;  // Space: O(k)
    private int size;     // Current number of elements
    private int front;    // Front pointer
    private int rear;     // Rear pointer

    public MyCircularQueue(int k) {  // Time: O(k), Space: O(k)
        queue = new int[k];
        size = 0;
        front = 0;
        rear = -1;  // Start at -1 for elegant first insert
    }

    public boolean enQueue(int value) {  // Time: O(1), Space: O(1)
        if (isFull()) return false;
        rear = (rear + 1) % queue.length;  // Circular increment
        queue[rear] = value;
        size++;
        return true;
    }

    public boolean deQueue() {  // Time: O(1), Space: O(1)
        if (isEmpty()) return false;
        front = (front + 1) % queue.length;  // Circular increment
        size--;  // Don't actually delete, just "forget"
        return true;
    }

    public int Front() {  // Time: O(1), Space: O(1)
        return isEmpty() ? -1 : queue[front];
    }

    public int Rear() {  // Time: O(1), Space: O(1)
        return isEmpty() ? -1 : queue[rear];
    }

    public boolean isEmpty() {  // Time: O(1), Space: O(1)
        return size == 0;
    }

    public boolean isFull() {  // Time: O(1), Space: O(1)
        return size == queue.length;
    }
}
```

Tips & Tricks

Key Insights:

- **Modulo magic**: (index + 1) % length creates circular behavior
- **size variable**: Solves empty/full ambiguity elegantly
- **rear = -1**: First enQueue becomes (-1+1)%k = 0 naturally
- **No data movement**: Just update pointers, O(1) operations
- **Overwriting**: Old values get overwritten when rear wraps around

Common Mistakes

- Without size: Can't distinguish empty (front==rear) from full
- rear++ instead of modulo → ArrayIndexOutOfBounds
- Clearing data on deQueue → Unnecessary, wastes time
- Starting rear at 0 → Need special case for first insert

Time & Space Complexity

Operation	Time	Space
Constructor	O(k)	O(k)
enQueue(value)	O(1)	O(1)
deQueue()	O(1)	O(1)
Front()	O(1)	O(1)
Rear()	O(1)	O(1)
isEmpty()	O(1)	O(1)
isFull()	O(1)	O(1)

Dry Run: MyCircularQueue(3), enQueue(1), enQueue(2), enQueue(3), deQueue(), enQueue(4)

Constructor: [_,_,_], f=0, r=-1, size=0

enQueue(1): r=(−1+1)%3=0
[1,_,_], f=0, r=0, size=1

enQueue(2): r=(0+1)%3=1
[1,2,_], f=0, r=1, size=2

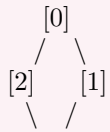
enQueue(3): r=(1+1)%3=2
[1,2,3], f=0, r=2, size=3 (FULL)

deQueue(): f=(0+1)%3=1
[1,2,3], f=1, r=2, size=2
(1 still in array but "invisible")

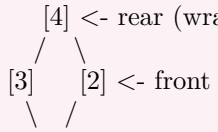
enQueue(4): r=(2+1)%3=0 (WRAPS!)
[4,2,3], f=1, r=0, size=3
(4 overwrites old 1)

Circular Visualization:

Array indices as circle (capacity = 3):



After enQueue(1,2,3), deQueue(), enQueue(4):



Logical queue order: [2, 3, 4]
Physical array: [4, 2, 3]

5 Implement Queue using Stacks (LC 232)

Problem: Implement FIFO queue using only LIFO stacks

Solution: Two Stacks with Lazy Transfer

```java
class MyQueue {
    private Stack<Integer> s1;  // Input stack, Space: O(n)
    private Stack<Integer> s2;  // Output stack, Space: O(n)

    public MyQueue() {  // Time: O(1), Space: O(1)
        s1 = new Stack<>();
        s2 = new Stack<>();
    }

    public void push(int x) {  // Time: O(1), Space: O(1)
        s1.push(x);  // Always push to s1
    }

    public int pop() {  // Time: O(1) amortized, O(n) worst, Space: O(1)
        if (s2.isEmpty()) {  // Lazy transfer only when needed
            while (!s1.isEmpty()) {
                s2.push(s1.pop());  // Transfer reverses order
            }
        }
        return s2.pop();
    }

    public int peek() {  // Time: O(1) amortized, O(n) worst, Space: O(1)
        if (s2.isEmpty()) {
            while (!s1.isEmpty()) {
                s2.push(s1.pop());
            }
        }
        return s2.peek();
    }

    public boolean empty() {  // Time: O(1), Space: O(1)
        return s1.isEmpty() && s2.isEmpty();
    }
}
```

Tips & Tricks

Key Insights:

- **Double reversal**: Stack reverses order; two stacks = original order
- **Lazy transfer**: Only move when s2 is empty (amortized O(1))
- **s1 = input, s2 = output**: Clear separation of concerns
- **Each element moves once**: Push to s1, transfer to s2, pop from s2 = 3 ops total

Common Mistakes

- Transferring on every operation → O(n) every time, not amortized O(1)
- Clearing s2 before transfer → Loses existing elements
- Not checking s2.isEmpty() → Breaks FIFO order
- return s1.isEmpty() || s2.isEmpty() → Wrong, should be AND

Time & Space Complexity

Amortized Analysis: Each element undergoes exactly 3 operations:

1. Push to s1: O(1)
2. Transfer to s2: O(1)
3. Pop from s2: O(1)

Total per element = O(3) = O(1) amortized
Dry Run: push(1), push(2), peek(), pop()

push(1): s1=[1], s2=[]
push(2): s1=[1,2], s2=[]
peek(): Transfer: s1=[], s2=[2,1]
return 1 (top of s2)
pop(): s2=[2,1] -> pop -> s2=[2]
return 1

Visualization:

After push(1,2):
s1 s2
|2| <-top | |
|1| | |
+-+ +-+

After peek() (transfer):
s1 s2
| | |1| <-top (front of queue!)
| | |2|
+-+ +-+

Reversal: s1 had [1,2] (2 on top)
s2 has [2,1] (1 on top) = Queue order!

6 Implement Stack using Queue (LC 225)

Problem: Implement LIFO stack using only FIFO queue

Solution: Single Queue with Rotation

```java
class MyStack {
    private Queue<Integer> q;  // Space: O(n)

    public MyStack() {  // Time: O(1), Space: O(1)
        q = new LinkedList<>();
    }

    public void push(int x) {  // Time: O(n), Space: O(1)
        q.offer(x);  // Add to rear
        // Rotate to bring new element to front
        for (int i = q.size() - 1; i > 0; i--) {
            q.offer(q.poll());  // Move front to rear
        }
    }

    public int pop() {  // Time: O(1), Space: O(1)
        return q.poll();  // Front is top of stack
    }

    public int top() {  // Time: O(1), Space: O(1)
        return q.peek();  // Front is top of stack
    }

    public boolean empty() {  // Time: O(1), Space: O(1)
        return q.isEmpty();
    }
}
```

Tips & Tricks

Key Insights:

- **Rotation trick**: After push, rotate (size-1) times to move new element to front
- **Expensive push**: O(n) to maintain stack order
- **Cheap pop/top**: O(1) because newest is always at front
- **Trade-off**: Opposite of queue-using-stacks (expensive write, cheap read)

Common Mistakes

- for (int i = q.size(); i > 0; i–) → Rotates n times (one extra)
- Not rotating → Queue behavior, not stack
- while (!q.isEmpty()) → Infinite loop!
- Rotating on pop → Wrong order, expensive pop

Time & Space Complexity

Dry Run: push(1), push(2), top(), pop()

```
push(1):   q=[1], no rotation (size-1=0)
push(2):   q=[1,2] -> rotate 1 time
           poll 1, offer 1 -> q=[2,1]
top():     return 2 (front)
pop():     q=[2,1] -> poll -> q=[1]
           return 2
```

Rotation Visualization:

push(3) to queue [1,2]:

Step 1: Add to rear
Front -> [1, 2, 3] <- Rear

Step 2: Rotate (size-1 = 2 times)

Rotation 1: poll 1, offer 1
Front -> [2, 3, 1] <- Rear

Rotation 2: poll 2, offer 2
Front -> [3, 2, 1] <- Rear

Result: Newest (3) is at front!
Stack view: [1, 2, 3] with 3 as top
Queue view: [3, 2, 1] with 3 at front

Quick Reference & Comparison

Complexity Comparison Table

Data Structure	Operation	Time	Space	Key Feature
4*Custom Stack	Constructor	O(maxSize)	O(maxSize)	Fixed size
	push	O(1)	O(1)	Check full
	pop	O(1)	O(1)	Check empty
	increment	O(min(k,n))	O(1)	Bottom k elements
3*LRU Cache	Constructor	O(1)	O(capacity)	HashMap + List
	get	O(1)	O(1)	Move to MRU
	put	O(1)	O(1)	Evict LRU
5*Min Stack (Pairs)	Constructor	O(1)	O(1)	Store [val, min]
	push	O(1)	O(1)	Track min
	pop	O(1)	O(1)	Auto-restore
	top	O(1)	O(1)	Access value
	getMin	O(1)	O(1)	Access min
5*Min Stack (Two)	Constructor	O(1)	O(1)	Parallel stacks
	push	O(1)	O(1)	Conditional push
	pop	O(1)	O(1)	Sync pop
	top	O(1)	O(1)	Main stack
	getMin	O(1)	O(1)	Min stack
7*Circular Queue	Constructor	O(k)	O(k)	Array allocation
	enQueue	O(1)	O(1)	Modulo wrap
	deQueue	O(1)	O(1)	Move front
	Front	O(1)	O(1)	Array access
	Rear	O(1)	O(1)	Array access
	isEmpty	O(1)	O(1)	size == 0
	isFull	O(1)	O(1)	size == capacity
4*Queue (2 Stacks)	Constructor	O(1)	O(1)	Two stacks
	push	O(1)	O(1)	Always s1
	pop	O(1) amortized	O(1)	Lazy transfer
	peek	O(1) amortized	O(1)	Lazy transfer
4*Stack (1 Queue)	Constructor	O(1)	O(1)	Single queue
	push	O(n)	O(1)	Rotate (n-1)
	pop	O(1)	O(1)	Front is top
	top	O(1)	O(1)	Front is top

Pattern Recognition

When to Use Each:
Array-Based (Custom Stack, Circular Queue):

- Fixed capacity requirement
- Need O(1) random access
- Memory locality important
- No dynamic resizing needed

HashMap + Linked List (LRU Cache):

- Need O(1) lookup AND insertion/deletion
- Order matters (LRU, MRU)
- Eviction policy required
- Key-value storage

Stack-Based (Min Stack):

- LIFO access pattern
- Need to track aggregate info (min, max, sum)
- Historical state tracking
- Recursive problem solving

Conversion Problems (Queue Stack):

- Understanding data structure properties
- Amortized analysis practice
- Trade-off analysis
- Interview fundamentals

Universal Tips

Design Patterns:

1. **Instance variables**: Use this. to distinguish from parameters
2. **Boundary checks**: Always check empty/full before operations
3. **Dummy nodes**: Simplify edge cases (no null checks)
4. **Modulo arithmetic**: Creates circular behavior elegantly
5. **Size tracking**: Solves ambiguity problems
6. **Lazy operations**: Defer expensive work until needed
7. **Space-time trade-off**: Store extra info for faster access

Common Pitfalls:

- Off-by-one errors (array indexing)
- Null pointer exceptions (always check)
- Using wrong comparison (< vs <=)
- Forgetting to sync multiple data structures
- Not handling duplicates correctly
- Premature optimization

Interview Strategies

Before Coding:

1. Clarify requirements (capacity? null values? duplicates?)
2. Ask about edge cases (empty? full? single element?)
3. Discuss time/space constraints
4. Draw the data structure
5. Explain your approach

While Coding:

1. Start with instance variables
2. Write constructor first
3. Implement helper methods
4. Handle edge cases explicitly
5. Add comments for complex logic
6. Think aloud

After Coding:

1. Trace through example
2. Test edge cases (empty, full, single element)
3. Analyze time/space complexity
4. Discuss optimizations
5. Mention alternative approaches

Follow-up Questions to Expect:

- "What if we need thread safety?"
- "How would you handle overflow?"
- "Can we optimize space?"
- "What if capacity is unknown?"
- "How does this scale?"

Memory Tricks

Remember These:
Custom Stack:

top = -1 (empty), increment bottom k

LRU Cache:

HashMap + DLL, Right=MRU, Left=LRU
Dummy nodes eliminate null checks

Min Stack:

Pairs: Store [val, min] together
Two: Use <= for duplicates, sync on pop

Circular Queue:

(index + 1) % capacity for wrap
rear = -1 initially, size tracks state

Queue (2 Stacks):

s1=input, s2=output, lazy transfer
Double reversal = original order

Stack (1 Queue):

Rotate (size-1) times after push
Newest element moves to front

Related Problems

Practice These Next:
Similar Design Problems:

- LC 705: Design HashSet
- LC 706: Design HashMap
- LC 380: Insert Delete GetRandom O(1)
- LC 460: LFU Cache (harder LRU variant)
- LC 641: Design Circular Deque
- LC 1670: Design Front Middle Back Queue

Stack Applications:

- LC 20: Valid Parentheses
- LC 739: Daily Temperatures
- LC 84: Largest Rectangle in Histogram
- LC 85: Maximal Rectangle

Queue Applications:

- LC 346: Moving Average from Data Stream
- LC 933: Number of Recent Calls
- LC 362: Design Hit Counter

Final Tips for Success

- **Master the fundamentals**: Understand WHY each solution works, not just HOW
- **Practice drawing**: Visualize data structures before coding
- **Start simple**: Get basic version working, then optimize
- **Test thoroughly**: Empty, single element, full capacity, duplicates
- **Explain clearly**: Interviewer wants to see your thought process
- **Know trade-offs**: Every design decision has pros and cons
- **Time management**: 5 min understand, 15 min code, 5 min test, 5 min optimize

Remember: These problems test design skills, not memorization. Focus on understanding patterns!